Қазақстан Республикасы Ғылым және жоғары білім министрлігі

Астана Халықаралық университеті

Жоба жұмыстары бойынша

ЕСЕП

Жоба тақырыбы: Өнімдерді автоматты тану жүйесі

Студент: Әуелхан Жанат Рустамұлы Топ: ВТиПО-24А ТиПО _____
/т.а.ж./ /қолы./

Студент: Тоқсанбай Балнұр Нұрашқызы Топ: ВТиПО-24А ТиПО _____
/т.а.ж./ /қолы./

Оқытушы Жолшиева Л.З. _____
/қолы./

Қорғалды «____»_____2025 ж. баға_____
/жазбаша./

Комиссия мүшелері: _____ _____
/т.а.ж./ /қолы./

_____ _____
/т.а.ж./ /қолы./

Астана, 2025 ж

Мазмұны

Кіріспе 2
1. Теориялық негіз 3
Жасанды нейрондық желілер және компьютерлік көру 3
CNN архитектурасы және Transfer Learning 3
Object Detection және YOLOv8 алгоритмі 4
Бағалау метрикалары 4
2. Бастапқы кезең 6
Жобаның алғашқы постановкасы 6
Датасет және оның дайындалуы 6
Transfer Learning процесі 7
CNN нәтижелері: Accuracy және Confusion Matrix 8
CNN жүйесінің шектеулері 8
3. Жетілдіру кезеңі 10
YOLOv8-ге көшу себептері 10
Аугментация стратегиясы және датасет бөлу 10
YOLOv8 моделін оқыту 13
4. Модель сапасын бағалау 15
Модельдерді салыстыру 15
CNN Confusion Matrix нәтижелері 15
5. Веб-платформа және пайдаланушы интерфейсі 17
Әрекеттер диаграммасы 17
Интерфейстің эволюциясы 18
Сурет жүктеу режимдері 19
6. Нәтижелер және талдау 21
7. Туындаған қиындықтар және шешімдер 24
Негізгі техникалық қиындықтар және олардың шешімдері 24
Біздің қосқан үлесіміз 24
Жобаны одан әрі дамыту бағыттары 25
Қорытынды 26
Пайдаланылған әдебиеттер 27
Қосымша А 28
Қосымша Б 34

Кіріспе

Қазіргі ғылым мен технологияның дамуы нәтижесінде жасанды интеллект «Artificial Intelligence, AI» және компьютерлік көру «Computer Vision» салалары тез қарқынмен дамып, адамдар өміріне тереңірек еніп жатыр. Бұл технологиялар тек зертханалық зерттеулермен шектелмей, нақты өмірдің алуан саласында — медицинада, көлік жүйелерінде, өнеркәсіпте, логистикада, бөлшек сауда мен ауыл шаруашылығында — кеңінен қолданылуда. Жасанды интеллект адам жұмысын алмастырып қана қоймай, оны жеңілдетіп, жылдамдатып, адам қателіктерінің алдын алуға мүмкіндік беруде.

Объектілерді тану «object recognition» және анықтау «object detection» мәселелері компьютерлік көру саласының негізгі бағыттарының бірі болып табылады. Бұл жүйелер адамның көзін алмастыра отырып, суреттер мен бейне ағынынан нысандарды жіктейді, олардың орналасуын анықтайды және нақты уақытта шешім қабылдайды. Заманауи нейрондық желілер технологиясы мұндай жүйелерді тек дәл ғана емес, сонымен бірге өте жылдам жасауға мүмкіндік берді.

Бөлшек сауда саласында, атап айтқанда Magnum сияқты ірі супермаркеттерде, тауарларды автоматты тану жүйелері өте маңызды практикалық мәнге ие. Мыңдаған позициядан тұратын тауар қорын адам күшімен басқару өте ауыр және қателікке бейім. Автоматтандырылған тану жүйелері кассадағы тауарды бірден анықтайды, қоймадағы тауар есебін жеңілдетеді, онлайн каталогтарды автоматты толықтырады және жалпы дүкен жұмыс процестерін оңтайландырады.

Осы жобаның мақсаты — азық-түлік өнімдерін автоматты түрде тану және детекциялау жүйесін жасау. Жоба екі кезеңнен тұрады: бастапқы кезеңде CNN «Convolutional Neural Network» архитектурасы негізінде классификация жүйесі жасалды, ал жетілдірілген кезеңде YOLOv8 архитектурасы негізінде толыққанды object detection жүйесі іске асырылды. Бұдан басқа, жүйеге нақты уақыт «real-time» режимінде жұмыс жасайтын веб-интерфейс қосылды.

Жобаның негізгі жетістіктері мынадай:

- 1-кезең: MobileNetV2 негізіндегі CNN классификациясы — 8 категория бойынша жоғары дәлдік «accuracy» метрикасымен тануды жүзеге асырады.
- 2-кезең: YOLOv8 негізіндегі object detection — bounding box арқылы нақты орналасуды анықтайды, нақты уақыт режимінде объектіні бір мезгілде детекциялайды.
- Веб-платформа: сурет жүктеу режимі мен нақты уақыт камера режимін қолдайтын толыққанды веб-интерфейс іске асырылды.

1. Теориялық негіз

1.1 Жасанды нейрондық желілер және компьютерлік көру

Нейрондық желілер «Neural Networks» — адам миының жұмыс принципінен шабыт алып жасалған есептеу модельдері. Олар өзара байланысқан «нейрондардан» тұрады және үлгілерді тануға, классификациялауға, болжам жасауға арналған. Биологиялық нейронға ұқсас жасанды нейрон белгілі бір кіріс сигналдарын қабылдайды, оларды өлшейді «weight», жинақтайды және белсендіру функциясы арқылы шығыс сигналын шығарады.

Жасанды нейрондық желілердің ерекше түрі — терең оқыту «Deep Learning» архитектуралары. Олар бірнеше жасырын қабаттан тұрып, деректердің иерархиялық ерекшеліктерін автоматты түрде үйренеді. Бастапқы қабаттар суреттен қарапайым белгілерді яғни, жиектерді, бұрыштарды анықтаса, тереңірек қабаттар күрделі пішіндерді, объектілерді тани бастайды. Суреттермен жұмыс жасайтын нейрондық желілердің ең тиімді архитектурасы — Convolutional Neural Networks «CNN».

Компьютерлік көру «Computer Vision» — компьютерлік жүйелердің суреттер мен бейнелерден мазмұнды «ұғып», соның негізінде шешім қабылдай алуын зерттейтін сала. Классикалық алгоритмдерде ерекшеліктерді адам қолмен анықтаса, терең оқыту архитектуралары бұл процесті толық автоматтандырады. Сондықтан заманауи компьютерлік көру жүйелері адам сарапшысымен тең немесе одан да жоғары дәлдікке жетуде.

1.2. CNN архитектурасы және Transfer Learning

CNN «Convolutional Neural Network» — суреттерді өңдеуге арналған арнайы нейрондық желі архитектурасы. Оның басты артықшылығы — пикселдер арасындағы кеңістіктік байланысты сақтай отырып, суреттен маңызды ерекшеліктерді иерархиялық түрде үйрене алуы. Дәстүрлі толық байланысты желілерден айырмашылығы — CNN суреттің жергілікті аймақтарын талдайды, бұл параметрлер санын айтарлықтай азайтады.

CNN-нің негізгі қабаттары және олардың рөлі:

- Convolutional Layer (Жинақтау қабаты) — арнайы сүзгілер «kernels/filters» арқылы суреттен ерекшеліктерді анықтайды. Бір қабатта бірнеше сүзгі параллель жұмыс жасайды, олардың әрқайсысы суреттің белгілі бір ерекшелігін — горизонталь/вертикаль жиектерді, бұрыштарды, текстураларды — анықтауға арналған.
- ReLU Activation (Белсендіру функциясы) — сызықтық емес трансформация жасайды. Нейрондық желінің күрделі байланыстарды меңгеруіне мүмкіндік береді. max(0, x) формуласымен жұмыс жасайды: теріс мәндерді нөлге ауыстырады.
- Pooling Layer (Жинақтау-қысқарту қабаты) — ерекшеліктер картасының өлшемін азайтады, маңызды ақпаратты сақтай отырып есептеу шығынын азайтады. Max Pooling — аймақтың ең үлкен мәнін алатын ең кең таралған тәсіл.
- Fully Connected Layer (Толық байланысты қабат) — алдыңғы қабаттардан алынған ерекшеліктерді біріктіріп, соңғы классификация шешімін қабылдайды.
- Softmax / Output Layer — шығыс қабатта класс ықтималдықтарын есептейді; барлық мәндердің қосындысы 1-ге тең болады.

Transfer Learning — бір тапсырма үшін оқытылған модельді екінші, соған ұқсас тапсырмада қолдану тәсілі. Бұл тәсіл аз деректермен де жоғары нәтиже беруге мүмкіндік

береді, өйткені модель бастапқы кезеңде жалпы визуалдық ерекшеліктерді «жиектер, текстуралар, формалар» үйреніп алған болады. MobileNetV2 ImageNet датасетінде 1 миллионнан астам сурет арқылы оқытылған — ол визуалдық ерекшеліктерді өте жақсы меңгерген, сондықтан оны тамақ өнімдерін жіктеу тапсырмасына бейімдеу оңай және тиімді болады.

Transfer learning іс жүзінде екі сатыда жүзеге асырылады: алдымен алдын ала оқытылған модель жүктеледі де соңғы классификация қабаттары алынып тасталады, орнына жаңа тапсырмаға арналған қабаттар қосылады. Бастапқыда тек осы жаңа қабаттар ғана оқытылады. Одан кейін fine-tuning кезеңінде базалық модельдің жоғарғы қабаттары да оқытуға қосылады, бұл жалпы дәлдікті айтарлықтай арттырады

1.3. Object Detection және YOLOv8 алгоритмі

Object Detection «Объектілерді анықтау» — суреттегі немесе бейнедегі объектілерді тауып, олардың класын анықтап, координаттарын «bounding box» беретін тапсырма. Классификациядан айырмашылығы — бір суретте бірнеше объект болуы мүмкін, жүйе олардың әрқайсысын жеке анықтайды. Бұл нақты дүкен ортасында өте маңызды, себебі бір камера кадрында ондаған тауар бір уақытта орналасуы мүмкін.

YOLO «You Only Look Once» — 2015 жылы ұсынылған, объектілерді анықтауға арналған революциялық алгоритм. Дәстүрлі object detection алгоритмдері R-CNN, Fast R-CNN суретті бірнеше рет талдаса, YOLO суретті бір рет қарап барлық объектілерді бір уақытта анықтайды. Бұл оны нақты уақыт тапсырмаларына өте қолайлы етеді. YOLO жұмыс принципі: сурет S×S торланған ұяшықтарға «grid cells» бөлінеді; әрбір ұяшық B bounding box болжайды; Non-Maximum Suppression «NMS» арқылы қабаттасқан bbox-тардың ішінен ең дәлі таңдалады. **YOLOv8 — Ultralytics компаниясы жасаған ең соңғы нұсқа (2023). Оның маңызды инновациялары:**

- Anchor-free detection — алдын ала анықталған якорь жиынтықтарын қолданбайды, бұл гиперпараметрлерді баптауды жеңілдетеді.
- CSPDarknet53 негізгі желісі — cross-stage partial connections арқылы ерекшеліктерді жоғары тиімділікпен алады.
- PANet «Path Aggregation Network» — әртүрлі өлшемдегі объектілерді тану үшін ерекшелік пирамидасы жасайды.
- Decoupled head — классификация және bbox регрессиясы үшін бөлек «бастар» қолданылады, бұл нақтылықты арттырады.
- Ultralytics Python API — оқыту, бағалау, экспорт және inference бірнеше жолдық кодпен жүзеге асырылады.

1.4. Бағалау метрикалары

Object detection жүйелерінің сапасын бағалауда классификациядан өзгеше метрикалар қолданылады. Бұл метрикаларды дұрыс түсіну — нәтижелерді дұрыс интерпретациялаудың негізі (1-кесте):

1-кесте. Бағалау метрикалары

Метрика	Анықтамасы	Формуласы
IoU	Болжанған bbox мен шынайы bbox арасындағы қабаттасу дәрежесі	(Area of Overlap) / (Area of Union)
Precision	Оң болжамдардың дәлдігі — жалған оң нәтижелердің аздығы	TP / (TP + FP)
Recall	Нақты объектілерді табу толықтығы — жіберіп алулардың аздығы	TP / (TP + FN)
F1-score	Precision мен Recall теңгерімді үйлесімі	2 × P × R / (P + R)
mAP@0.5	IoU≥0.5 шартындағы орташа дәлдік (негізгі метрика)	mean(AP) барлық кластар бойынша
mAP@0.5:0.95	COCO стандарты — ең қатаң жалпы метрика	Орташа mAP (IoU 0.5–0.95)

mAP@0.5 бойынша бағалау шкаласы:

- 0.5 жоғары — минималды қабылданатын нәтиже;
- 0.75+ — жақсы;
- 0.85+ — өте жақсы;
- 0.90+ — кәсіби деңгей.

2. Бастапқы кезең

2.1 Жобаның алғашқы постановкасы

Жобаның бастапқы кезеңінде мақсат қарапайым болды: берілген суреттен оның қай азық-түлік категориясына жататынын анықтайтын жүйе жасау. Бұл — классификация тапсырмасы, яғни «бір сурет — бір класс» принципі. Осы кезеңде шешілуі тиіс негізгі сұрақтар:

- Жүйе суретті 8 категорияның біріне дұрыс жіктей ала ма?
- Transfer learning тәсілі аз деректермен де жақсы нәтиже бере ме?
- Модельдің дәлдігі нақты өмір жағдайларына жеткілікті ме?

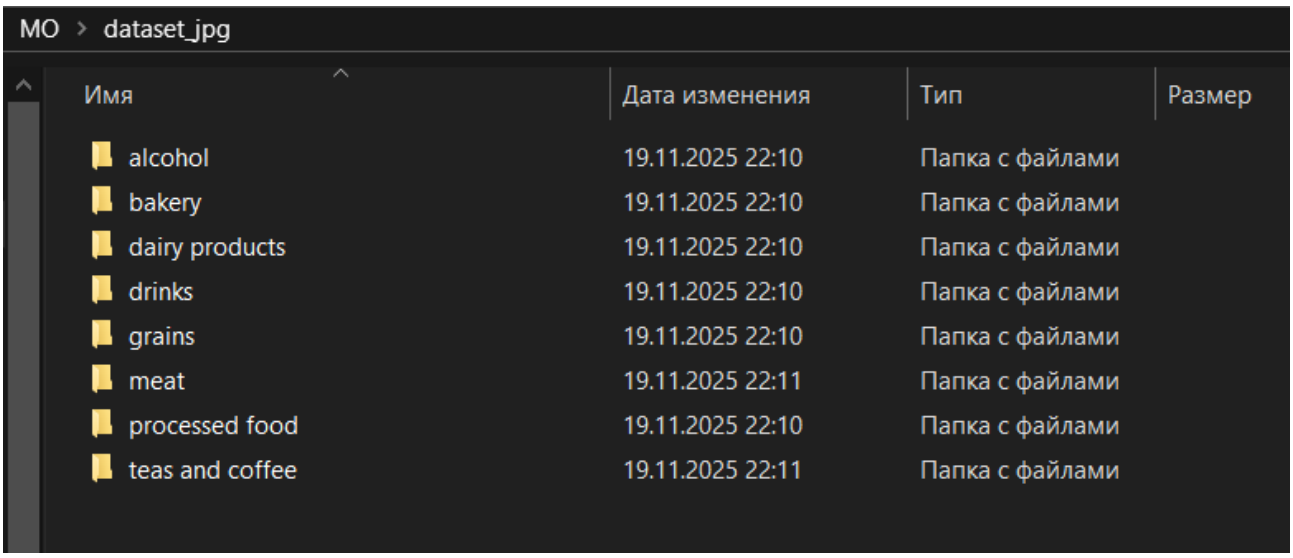
CNN архитектурасы ретінде MobileNetV2 таңдалды. Бұл таңдауды негіздейтін бірнеше себеп бар:

- Жеңілдік және жылдамдық: MobileNetV2 мобильді және енгізілген «embedded» жүйелерге арналған, өте аз параметр санымен жоғары нәтиже береді. VGG16 немесе ResNet50 сияқты ауыр архитектуралармен салыстырғанда бірнеше есе жылдам жұмыс жасайды.
- Depthwise Separable Convolutions: дәстүрлі конволюцияның орнына қолданылып, есептеу шығынын 8-9 есе азайтады, дәлдікті сәл ғана төмендетеді.
- Inverted Residuals: MobileNetV2-нің басты инновациясы — ақпаратты тиімдірек өткізіп, gradient flow-ды жақсартатын inverted residual блоктары.
- ImageNet алдын ала оқыту: Модель 14 миллионнан астам суреттен тұратын ImageNet датасетінде оқытылған, бұл оған визуалдық ерекшеліктерді өте жақсы меңгертеді.
- Ашық лицензия: TensorFlow/Keras кітапханасында еркін қолжетімді, коммерциялық қолдану үшін де рұқсатты.

2.2 Датасет және оның дайындалуы

Классификация моделін оқыту үшін 8 категориядан тұратын датасет жиналды (2.2 сурет). Барлық суреттер Magnum супермаркетінде тікелей түсірілді — бұл деректердің шынайы ортаға сәйкестігін қамтамасыз етті. Датасет папкалық иерархия форматында ұйымдастырылды:

- Alcohol — ішімдіктер мен алкогольді сусындар
- Bakery — наубайхана өнімдері: нан, тоқаш, печенье
- Dairy products — сүт өнімдері: сүт, кефир, йогурт, ірімшік
- Drinks — алкогольсіз сусындар: шырын, газды су, компот
- Grains — дән өнімдері: жарма, ұн, макарон, күріш
- Meat — ет өнімдері: жас ет, шұжық, тауық еті
- Processed food — өңделген тағамдар: консервілер, дайын тағамдар
- Teas and coffee — шай мен кофе өнімдері



Сурет 2.2 – Dataset

Dataset дайындалу кезінде мынадай критерийлер ескерілді:

- суреттер алуан фон, жарықтық деңгейі және бұрыш жағдайларында болуы тиіс;
- класс балансы сақталуы керек;
- суреттер кем дегенде 224×224 пиксель өлшемінде болуы қажет.

Деректерді алдын ала өңдеу кезеңінде:

- барлық суреттер 224×224 пиксель өлшеміне келтірілді MobileNetV2 стандарты;
- пиксель мәндері 0–255 диапазонынан 0–1 аралығына нормализацияланды;
- датасет оқыту (80%) және валидация (20%) жиындарына бөлінді;
- prefetch() операциясы арқылы оқу өнімділігі жақсартылды.

Dataset көлемін арттыру үшін data augmentation тәсілдері қолданылды:

- суреттерді айналдыру «rotation_range=30°»;
- горизонталь және вертикаль бағытта жылжыту;
- масштабтау «zoom_range)»;
- айналық көрсету «horizontal_flip» және жарықтықты өзгерту. Бұл тәсіл модельдің overfitting-ін азайтады және деректерді жасанды түрде көбейтеді.

Әр бастапқы суреттен 20 жаңа нұсқа генерацияланды.

2.3 Transfer Learning процесі

MobileNetV2 transfer learning негізіндегі жүйенің архитектурасы келесі қабаттардан тұрады:

1. Кіріс қабаты «Input Layer»: 224×224×3 өлшемді RGB суреттер қабылдайды.
2. MobileNetV2 Базалық Моделі: ImageNet-те алдын ала оқытылған негізгі бөлік. Кіріс суреттен маңызды визуалды ерекшеліктерді алады. Бастапқыда мұздатылған «trainable=False», fine-tuning кезінде соңғы 120 қабат ашылды.
3. Global Average Pooling «GAP»: Feature map-ты 1280 элементті бір өлшемді векторға түрлендіреді. Параметрлер санын азайтып, модельді тиімдірек етеді.
4. Dropout (0.3): Overfitting болдырмау үшін оқыту кезінде нейрондардың 30%-ын кездейсоқ өшіреді.

5. Dense Output Layer: 8 нейроннан тұрады (8 категория үшін). Softmax функциясы арқылы ықтималдықтар есептейді.

Модельді оқыту екі негізгі кезеңнен тұрды. Бірінші кезеңде «10 эпоха» MobileNetV2-нің барлық негізгі қабаттары мұздатылып, тек жаңа classifier head оқытылды. Екінші кезеңде «15 эпоха, fine-tuning» соңғы 120 қабат ашылып, learning rate 1e-5-ке дейін төмендетілді. Деректер теңсіздігін азайту үшін compute_class_weight арқылы класс салмақтары есептеліп, оқытуда қолданылды. Оқыту параметрлері:

- Optimizer — Adam;
- Loss — categorical_crossentropy;
- Batch size — 32.

2.4 CNN нәтижелері: Accuracy және Confusion Matrix

Оқыту барысында 3367 сурет, валидация үшін 594 сурет қолданылды. 25 эпохадан кейін алынған нәтижелер:

2-кесте. Оқыту нәтижелері

Көрсеткіш	**Оқыту (Training)**	**Валидация (Validation)**
Дәлдік (Accuracy)	98.93%	96.97%
Шығын (Loss)	0.0223	0.0730

Кестеден көріп отырғанымыздай, модельдің валидациялық дәлдігі 96.97%-ды құрайды. Бұл нәтиже жобаның басында қойылған 90% дәлдік мақсатын айтарлықтай асыра орындағанын көрсетеді. Training және validation accuracy мәндерінің жақындығы overfitting мәселесінің жоқтығын дәлелдейді.

Confusion Matrix бойынша Alcohol және Teas and coffee категориялары ең жоғары дәлдікпен танылды (98%), ал Bakery мен Dairy products категорияларында шамалы шатасулар байқалды (3–5%). Processed food кейде Meat категориясымен 6%-ға шатасты. Бұл нәтижелер ұқсас сыртқы пішіні бар тауарларды ажырату үшін датасетті одан әрі толықтыру қажеттілігін көрсетеді.

2.5 CNN жүйесінің шектеулері

Бастапқы CNN классификация жүйесі өз тапсырмасын орындады, алайда жоба дамыған сайын бірқатар маңызды шектеулер анықталды:

- Жүйе бүтін суретті бір класқа жатқызады — бір суретте екі немесе одан да көп тауар болса, жүйе тек біреуін «таниды».
- Объектінің суреттегі орналасуы (координаттары) белгісіз болып қалады — жүйе «нан бар» дейді, бірақ оның нақты орнын нұсқай алмайды.

- Тауар суреттің шетінде немесе фонның бір бөлігі болса, жіктеу дәлдігі айтарлықтай төмендейді.
- Нақты уақыт режимінде пайдалану қиын: бейне ағынының кадрларында бірнеше объект болуы мүмкін.
- Жаңа тауарды тануға үйрету үшін барлық деректерді қайта оқыту қажет.

Осы шектеулерді жою үшін жобаның екінші кезеңінде YOLOv8 object detection архитектурасына өту шешімі қабылданды

3. Жетілдіру кезеңі

3.1. YOLOv8-ге көшу себептері

CNN классификация жүйесінің шектеулерін талдай келе, жобаны object detection бағытына ауыстыру шешімі қабылданды. Нақты дүкен ортасында бір суретте немесе камера кадрында бір уақытта бірнеше тауар болуы — бұл CNN үшін шешілмейтін мәселе. Тауар белгілеу, автоматтандырылған касса немесе қойма есебі сияқты қосымшаларда объектінің нақты орналасуы міндетті түрде қажет. Мобильді немесе веб-қосымшада камера арқылы нақты уақытта тауарды тану — заманауи тұтынушылар күтетін мүмкіндік.

Object detection үшін CNN-дегідей жай папкаларға бөлінген суреттер жеткіліксіз. Әрбір суретте объектінің нақты орналасуын сипаттайтын аннотация файлы болуы керек. YOLO аннотация форматы: әрбір сурет үшін бір .txt файл жасалады, онда суреттегі әр объект үшін class_id, x_center, y_center, width, height — бес мән беріледі, координаттар сурет өлшемімен нормаланған (0.0–1.0 аралығында).

Бұрынғы CNN датасетінде тек класс белгісі болса, YOLOv8 үшін барлық суреттерге bounding box аннотациясы Roboflow платформасы арқылы жасалды. Roboflow мынадай мүмкіндіктер ұсынады:

- интерактивті аннотация редакторы,
- Smart Auto-Label функциясы,
- аугментация конвейері (bbox координаттары да автоматты жаңартылады),
- YOLO/COCO/VOC форматтарына экспорт, нұсқа бақылау (versioning).

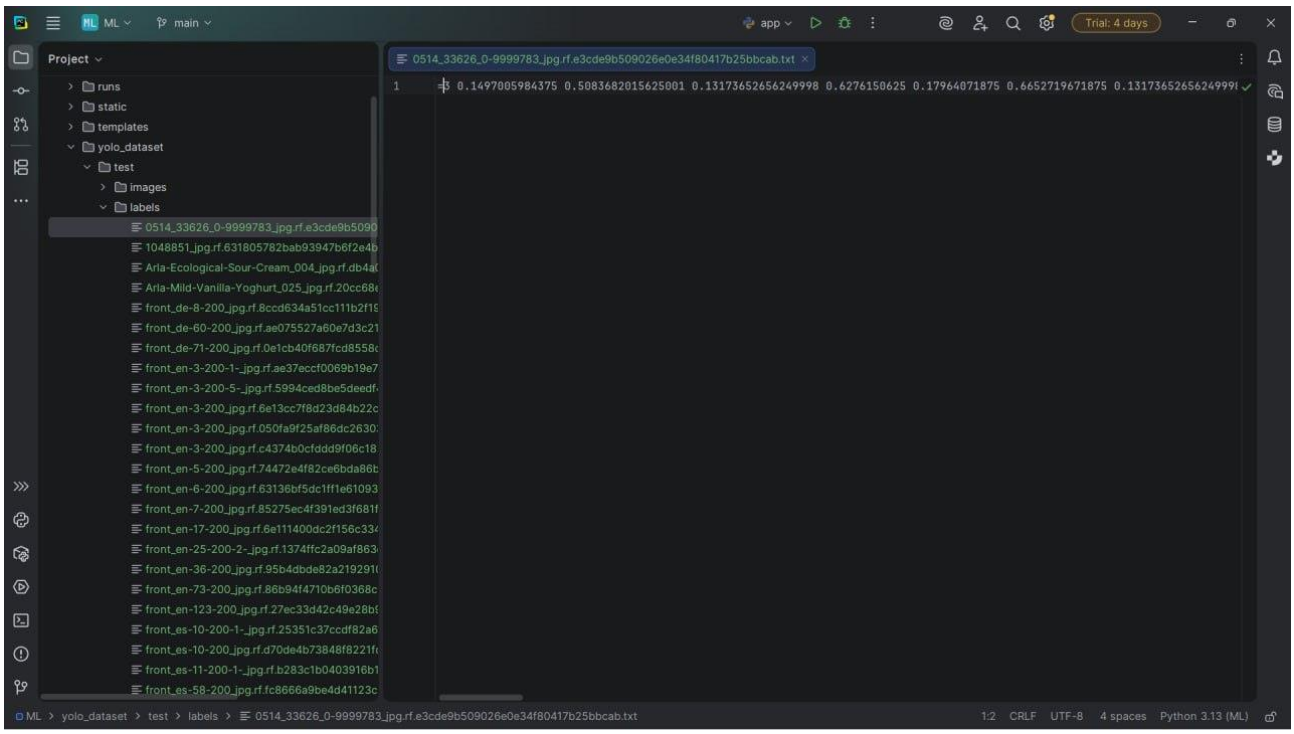
3.2. Аугментация стратегиясы және датасет бөлу

Object detection үшін CNN-дегідей жай папкаларға бөлінген суреттер жеткіліксіз. Әрбір суретте объектінің нақты орналасуын сипаттайтын аннотация файлы болуы керек. YOLO аннотация форматы: әрбір сурет үшін бір .txt файл жасалады (3.2.1 сурет), онда суреттегі әр объект үшін бір жол болады:

<class_id> <x_center> <y_center> <width> <height>

Мұнда:

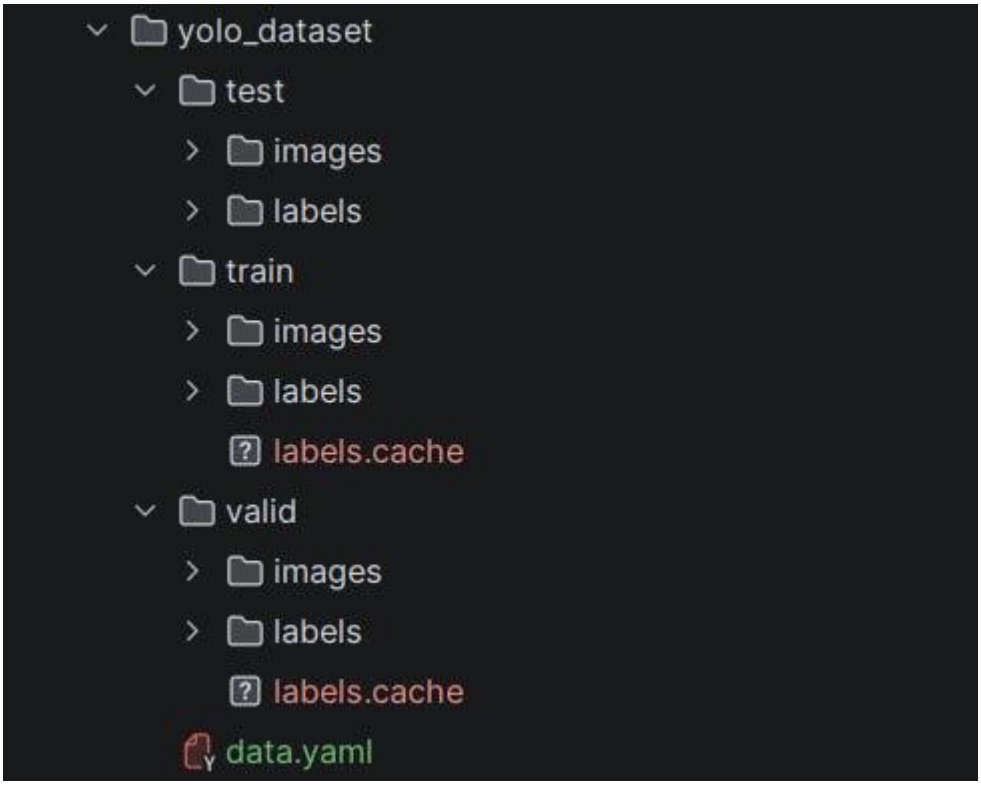
- class_id — класс нөмірі (0-ден 7-ге дейін, 8 класс үшін).
- x_center, y_center — bounding box-тің ортасының координаттары, сурет өлшемімен нормаланған (0.0 – 1.0 аралығында).
- width, height — bounding box-тің ені мен биіктігі, нормаланған.



Сурет 3.2.1 – YOLO labels

Мысал: «0 0.512 0.488 0.301 0.620» — бұл alcohol класының объектісі, суреттің ортасына жуық орналасқан, биіктігі суреттің 62%-ын алып жатыр дегенді білдіреді.

Датасет папкалық құрылымы YOLO стандартына сәйкес ұйымдастырылды (3.2.2 сурет):



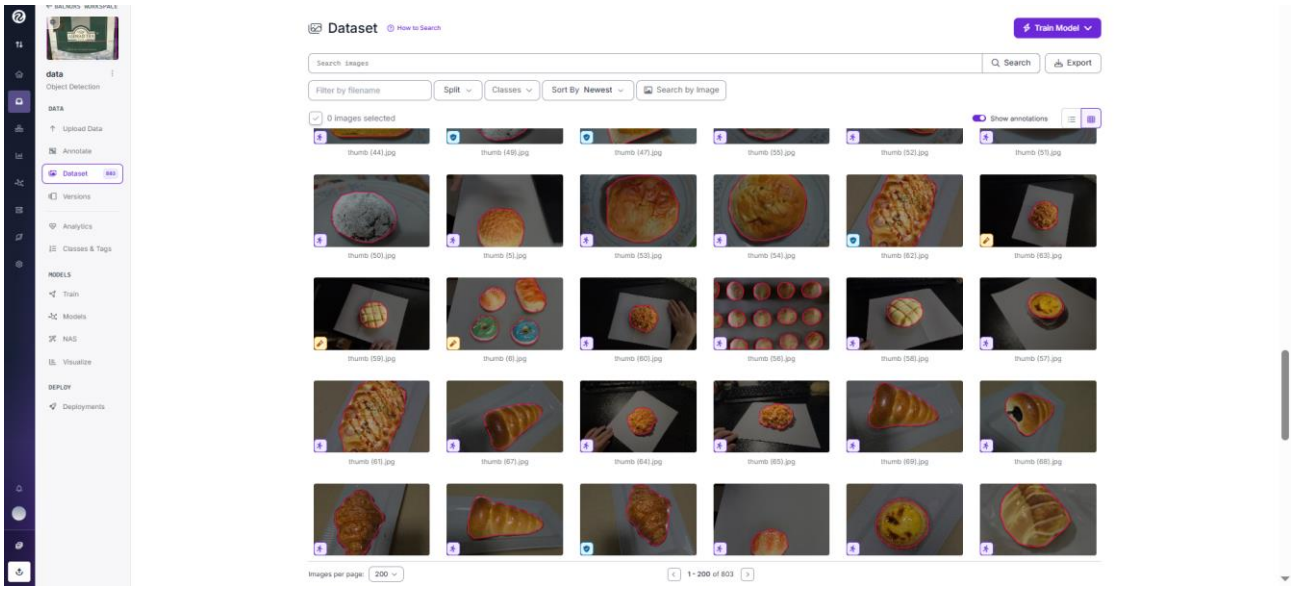
Сурет 3.2.2 – Жаңартылған Dataset

Бұрынғы CNN датасетінде тек класс белгісі болса, YOLOv8 үшін барлық суреттерге bounding box аннотациясы жасалды. Бұл жұмыс Roboflow платформасы арқылы жүргізілді (3.2.3 сурет).

Roboflow — компьютерлік көру датасеттерін жасауға, өңдеуге және басқаруға арналған онлайн платформа. Ол мынадай мүмкіндіктер ұсынады:

- Суреттерді жаппай жүктеу және ұйымдастыру — Google Drive, Dropbox немесе тікелей жүктеу арқылы.
- Интерактивті аннотация редакторы — тінтуірмен bounding box сызу, классты тағайындау, өзгерту.
- Smart Auto-Label — AI арқылы аннотацияны жартылай автоматтандыру.
- Аугментация конвейері — аннотациялармен бірге суреттерді трансформациялау, bbox координаттары да автоматты түрде жаңартылады.
- Форматтауды экспорттау — YOLO, COCO, VOC Pascal және басқа форматтарға бір рет басу арқылы экспорттау.
- Нұсқа бақылау «versioning» — датасеттің әртүрлі нұсқаларын сақтау мүмкіндігі.

Roboflow-да аннотация жасаудың артықшылығы — суреттерді файл жүйесінде қолмен белгілеп, координаттарды өлшеу қажет емес. Платформа бұл процесті айтарлықтай жеделдетеді.



Сурет 3.2.3 – Roboflow жүйесі

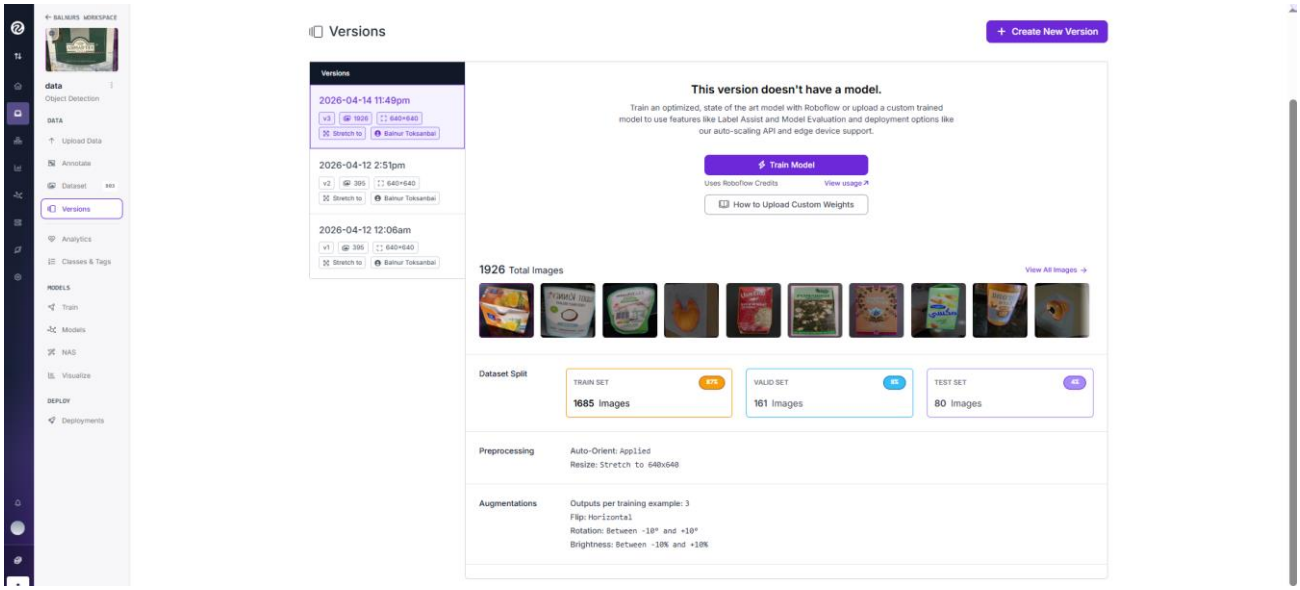
Датасеттің көлемі мен алуантүрлілігін арттыру үшін аугментация — деректерді жасанды көбейту тәсілі — қолданылды (3.2.4 сурет). Аугментацияның маңыздылығы: нейрондық желі нақты өмірде тауарларды алуан жарық жағдайында, бұрышта, ракурста кездестіреді, сондықтан модель осы алуантүрлілікке бейімделген болуы тиіс.

Қолданылған аугментация түрлері:

- Айналдыру «Rotation ±10°»: Тауарлар сөрелерде қисаюы немесе суреттер қисаң түсірілуі мүмкін.
- Горизонталь айна эффектісі «Horizontal Flip»: Тауардың оң немесе сол жақта орналасуы — бірдей класс.

- Жарықтық өзгерту «Brightness ±10%»: Дүкендерде жарықтандыру деңгейі әртүрлі болады.

Аугментацияның маңызды ерекшелігі — Roboflow-да трансформация жасалғанда bounding box координаттары да автоматты түрде жаңартылады. Яғни суретті айналдырғанда bbox-тың жаңа координаттары дұрыс есептеліп, .txt файлда жаңартылады.



Сурет 3.2.4 – Roboflow жүйесіндегі модель

Датасет үш бөлікке стандартты пропорцияда бөлінді (3.2.4 сурет):

- Train жиыны (70%): Модельді оқыту үшін. Модель осы деректерден үйренеді.
- Validation жиыны (20%): Оқыту барысында гиперпараметрлерді баптауға және overfitting-ді бақылауға қолданылады.
- Test жиыны (10%): Соңғы бағалау үшін. Бұл деректер оқыту кезінде мүлде пайдаланылмайды.

Бөлу кезінде класстардың пропорционалды таралуы stratified split сақталды, яғни 70/20/10 пропорция барлық 8 класс үшін жеке сақталады.

3.3. YOLOv8 моделін оқыту

YOLOv8 моделі Ultralytics кітапханасы арқылы оқытылды (3.3 сурет). Оқыту конфигурациясы мынадай болды:

- Базалық модель: YOLOv8n «nano» — ең жеңіл нұсқа, inference жылдамдығы жоғары.
- Epochs: 50 — модель датасетті 50 рет толық қарайды.
- Image size: 640×640 — YOLOv8-нің стандартты кіріс өлшемі.
- Batch size: 8 — бір итерацияда 8 сурет өңделеді.
- Optimizer: AdamW автоматты таңдалады.
- Learning rate: косинустік жылдамдық жоспарымен «cosine LR schedule» автоматты реттелді.

Оқыту барысында Ultralytics автоматты түрде validation деректеріндегі mAP мәнін бақылайды. Егер validation mAP бірнеше epoch бойына жақсармаса, early stopping іске қосылып, overfitting-

тің алдын алады. YOLOv8-дің ең жылдам inference үшін nano нұсқасы таңдалды — ол CPU-де де нақты уақыт режимін қамтамасыз ете алады.

train_yolo.py

```python
from ultralytics import YOLO

model = YOLO("yolov8n.pt")

model.train(
    data="yolo_dataset/data.yaml",
    epochs=50,
    imgsz=640,
    batch=8
)

print("Обучение завершено")
```

Сурет 3.3 – Модельді оқыту

4. Модель сапасын бағалау

4.1. Модельдерді салыстыру

Жобаның екі кезеңіндегі модельдерді жан-жақты салыстыру жүйенің эволюциясын нақты суреттейді (3-кесте):

3-кесте. Модельдерді салыстыру

Критерий	**CNN (MobileNetV2)**	**YOLOv8**
Тапсырма	Классификация	Detection + Классификация
Bounding Box анықтау	Жоқ	Бар
Бірнеше объект анықтау	Жоқ	Бар
Нақты уақыт режимі	Жоқ	Бар (Real-Time)
Датасет форматы	Папкалар (класс атауы)	Images + Labels (.txt)
Аннотация күрделілігі	Төмен	Орташа
Оқыту жылдамдығы	Жылдам	Орташа
Inference жылдамдығы	Өте жылдам	Жылдам
Негізгі метрика	Accuracy	mAP, Precision, Recall
Практикалық қолдану	Шектеулі	Жоғары

4.2. CNN Confusion Matrix нәтижелері

4-кесте. CNN Confusion Matrix

Категория	**Alcohol**	**Bakery**	**Meat**	**Dairy**	**Grains**	**Drinks**	**Proc.food**	**Teas/Coffee**
Alcohol	98	0	0	0	0	2	0	0
Bakery	0	93	1	3	0	0	3	0
Meat	0	2	90	0	0	0	8	0
Dairy products	0	5	0	92	0	0	3	0
Grains	0	0	0	0	95	0	5	0

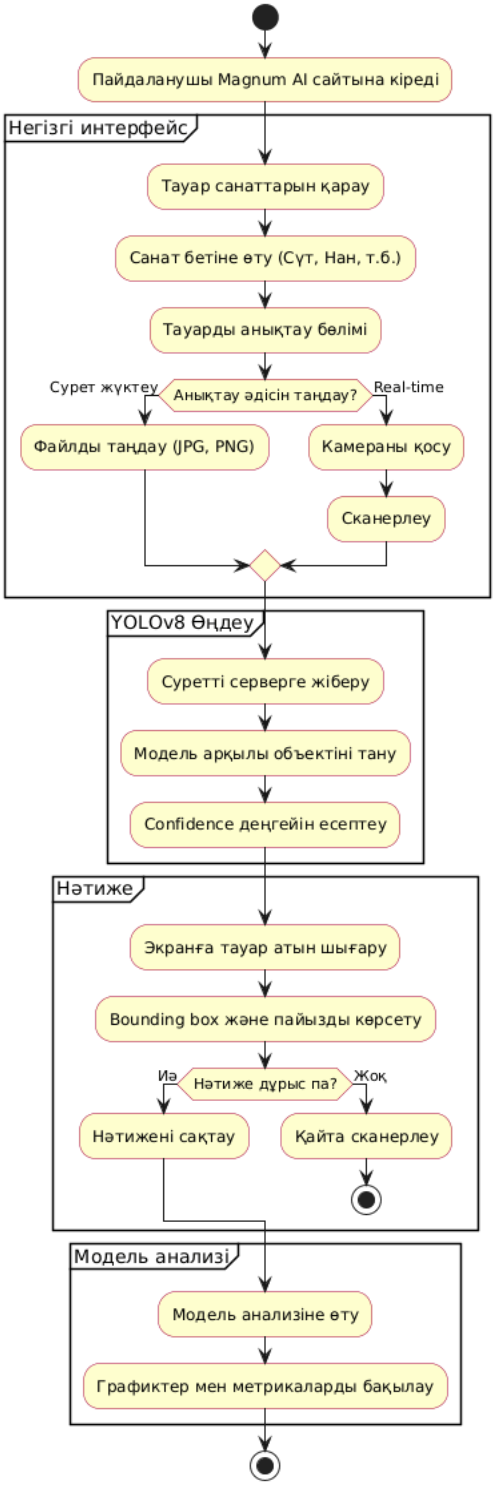
Категория	Alcohol	Bakery	Meat	Dairy	Grains	Drinks	Proc.food	Teas/Coffee
Drinks	1	0	0	0	0	97	0	2
Processed food	0	3	6	1	0	0	90	0
Teas and coffee	0	0	0	0	0	2	0	98

Confusion matrix талдауы бойынша: Alcohol және Teas and coffee категориялары 98% дәлдікпен танылды. Ұқсас сыртқы пішіні бар Processed food мен Meat арасында шатасу 6%-ға жетті. Бакалея мен сүт өнімдерінің оралымдарының ұқсастығынан Bakery және Dairy products арасында 3–5% қателік байқалды. Бұл нәтижелер датасетті арнайы толықтыру және fine-tuning арқылы жақсартуға болатынын көрсетеді.

5. Веб-платформа және пайдаланушы интерфейсі

5.1. Әрекеттер диаграммасы

Әрекеттер диаграммасы жүйенің жұмыс істеу процесін, яғни пайдаланушы суретті жүктеген сәттен бастап, нәтижені алғанға дейінгі алгоритмдік қадамдарды визуализациялау үшін қолданылады. Бұл жобада әрекеттер диаграммасы веб-қосымша мен нейрондық желінің өзара байланысын сипаттайды (5.1 сурет).



Сурет 5.1 - UML Activity Diagram

Жүйенің жұмыс істеу алгоритмі келесі қадамдардан тұрады:

1. Жүйеге кіру - Start
   - Пайдаланушы Magnum AI веб-сайтына кіреді
   - Жүйемен алғашқы өзара әрекет басталады
2. Негізгі интерфейс - User Interface
   - Пайдаланушы тауар санаттарын қарайды
   - Қажетті санатты таңдап, оның бетіне өтеді мысалы: сүт өнімдері, нан өнімдері және т.б.
   - Одан кейін тауарды анықтау бөліміне өтеді
3. Анықтау әдісін таңдау

   1-жол: Сурет жүктеу
   - Пайдаланушы дайын суретті таңдайды (JPG, PNG форматтары)
   - Бұл әдіс статикалық кескіндермен жұмыс істеуге арналған

   2-жол: Real-time анықтау
   - Пайдаланушы құрылғы камерасын іске қосады
   - Тауарды тікелей камера арқылы сканерлейді
4. YOLOv8 өңдеу кезеңі
   - Жүктелген немесе сканерленген сурет серверге жіберіледі
   - Сурет арнайы оқытылған YOLOv8 моделі арқылы өңделеді
   - Модель объектіні анықтап, оған қатысты сенімділік деңгейін confidence есептейді
5. Нәтиже көрсету - Output
   - Анықталған тауардың атауы экранға шығарылады
   - Тауардың орналасқан жері bounding box арқылы көрсетіледі
   - Сонымен қатар анықтау дәлдігі пайызбен беріледі

Одан кейін жүйе нәтижені тексеруді ұсынады:

- Егер нәтиже дұрыс болса - мәлімет сақталады
- Егер қате болса - пайдаланушы қайта сканерлеу мүмкіндігіне ие болады

6. Модель анализі - Model Analysis
   - Модельдің жұмыс тиімділігі қарастырылады
   - Әртүрлі графиктер мен метрикалар көрсетіледі
   - Бұл бөлім модельдің қаншалықты дәл жұмыс істейтінін бағалауға мүмкіндік береді
7. Аяқталу - Stop
   - Жүйенің толық жұмыс циклі аяқталғанын білдіреді
   - Пайдаланушы барлық қажетті ақпаратты алады

5.2. Интерфейстің эволюциясы

Бастапқы CNN кезеңінде веб-интерфейс тек сурет жүктеу және классификация нәтижесін мәтін түрінде шығару мүмкіндігін берді. Жүйе YOLOv8-ге ауысқаннан кейін интерфейс де түбегейлі жаңартылды. Жаңа интерфейстің негізгі принципі — пайдаланушыға бейімделген, интуитивті, екі режимді жұмыс тәртібі.

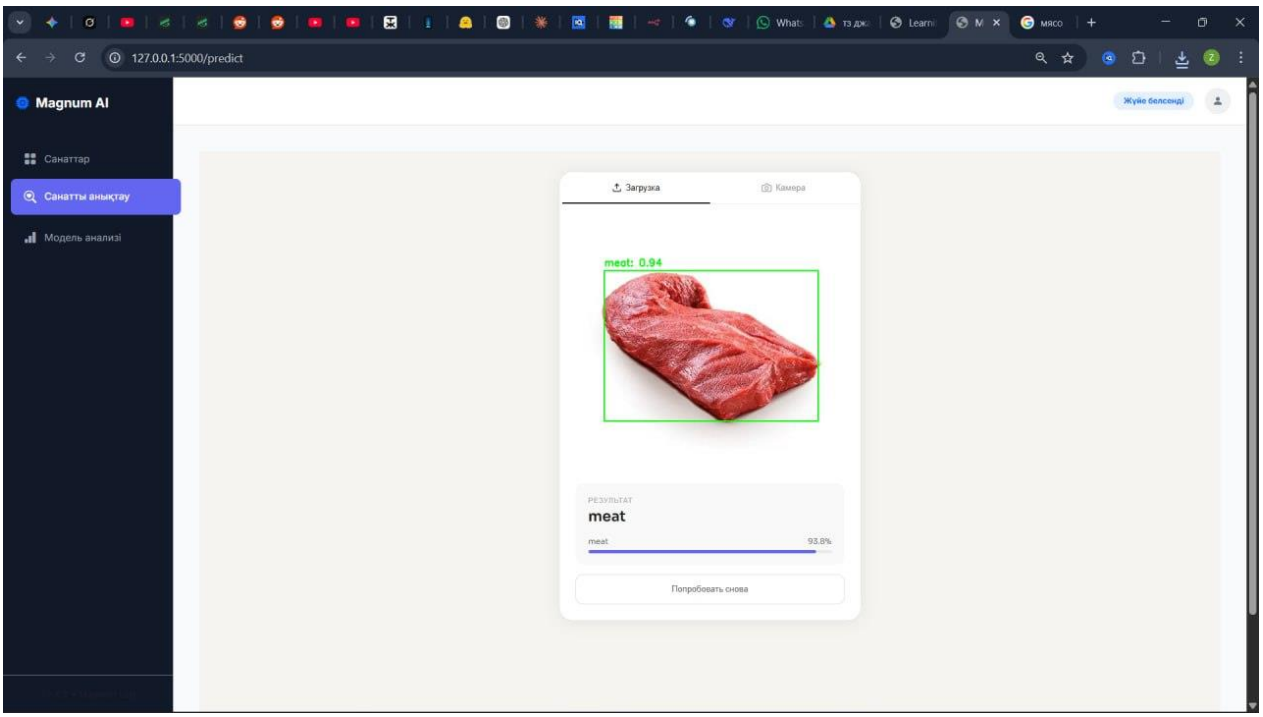
Веб-платформаның техникалық стегі мынадай компоненттерден тұрады:

- Backend: Python «Flask/FastAPI» — YOLOv8 inference логикасы серверде орындалады.
- Frontend: HTML5 + CSS3 + JavaScript — пайдаланушы интерфейсі, адаптивті дизайн.
- Camera API: WebRTC / MediaDevices API — браузерден камера ағынын алу.

- Real-Time байланыс: WebSocket немесе Server-Sent Events — сервер мен клиент арасындағы жылдам байланыс.
- Model: YOLOv8n «best.pt» — оқытылған модель сервер жадында үнемі жүктелген тұрады.

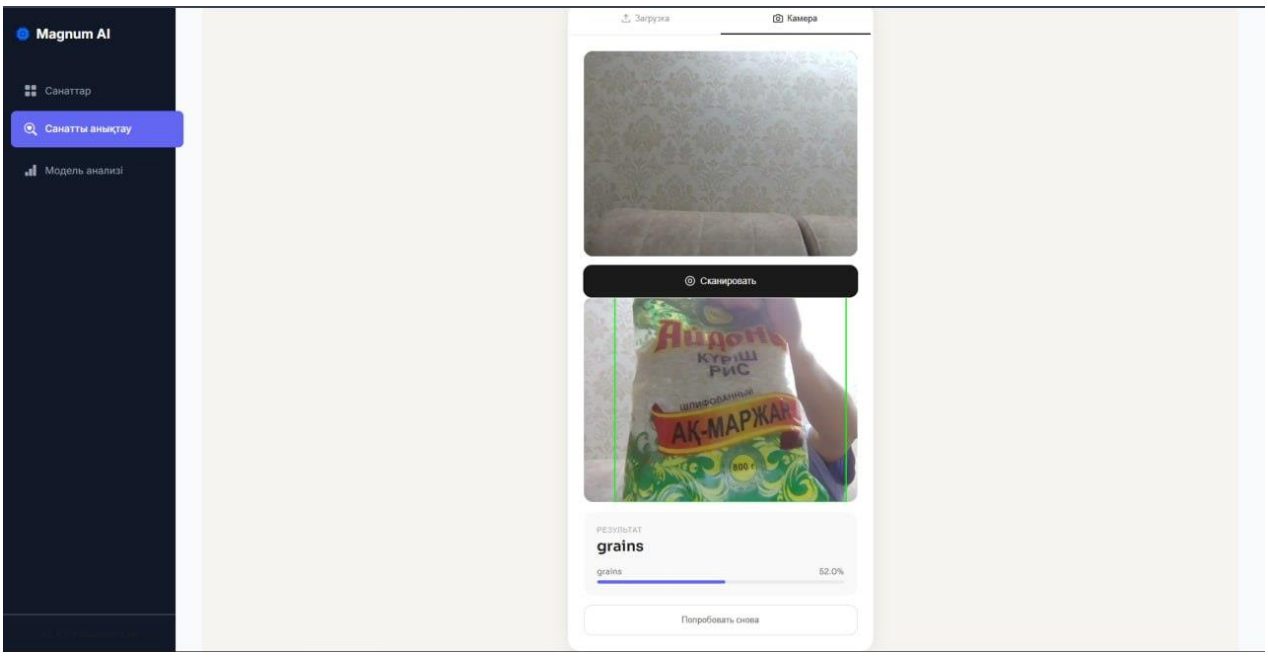
5.3. Сурет жүктеу режимдері

Сурет жүктеу режимінде пайдаланушы кез-келген суретті (JPEG, PNG, WebP) жүктейді (5.3.1 сурет). Жүйе суретті YOLOv8 арқылы талдайды. Табылған әрбір объект bounding box түрлі-түсті рамка арқылы суретте белгіленеді. Рамканың үстінде класс атауы және confidence score % жазылады. Бір суретте бірнеше объект болса, барлығы бір уақытта белгіленеді. Нәтиже сурет ретінде немесе JSON форматында жүктеп алуға болады.



Сурет 5.3.1 - Image Upload Mode

Нақты уақыт режимі — жобаның ең маңызды жаңа функциясы. Пайдаланушының браузері арқылы камерасына рұқсат сұралады. Камера бейнесі үздіксіз ағынмен сервер немесе клиент тарапына жіберіледі. Әрбір кадр YOLOv8 моделі арқылы өңделеді. Табылған тауарлар bounding box-пен тікелей камера бейнесінде белгіленеді. Пайдаланушы экранда нақты уақытта тауар атауларын, санын және орналасуын көреді (5.3.2 сурет).



Сурет 5.3.2 - Real-Time Detection Mode

Нақты уақыт режимінің практикалық қолданыс бағыттары:

- Супермаркет кассасы: тауарды сканерсіз автоматты тану.
- Қойма есебі: сөредегі тауарларды автоматты санау мен тіркеу.
- Мобильді шоппинг: камераны тауарға бағыттап, толық ақпарат алу.
- Тамақ журналы: жеген тамақты суретке түсіріп, автоматты жазу.

6. Нәтижелер және талдау

YOLOv8 моделінің нысандарды тану дәлдігін тексеру үшін келесі негізгі метрикалар қолданылды (6 сурет):

1. Precision (Дәлдік). Бұл метрика модельдің табылған нысандар ішінде қаншалықты қате жібермегенін көрсетеді. Яғни, модель "бұл нан" деп тапса, оның шынымен нан болу ықтималдығы:

$$Precision = \frac{TP}{TP+FP}$$

Мұндағы TP — дұрыс табылғандар, FP — қате табылғандар.

2. Recall (Толықтық). Модельдің суреттегі барлық нысандарды қаншалықты толық тапқанын көрсетеді:

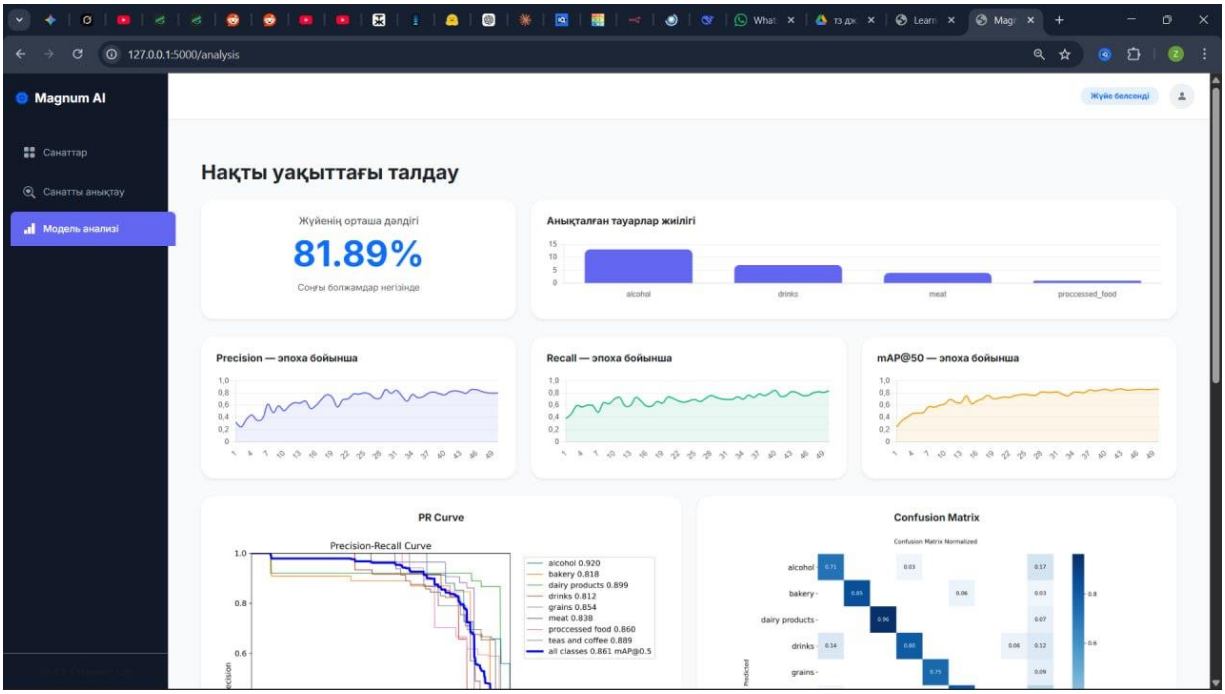
$$Recall = \frac{TP}{TP+FN}$$

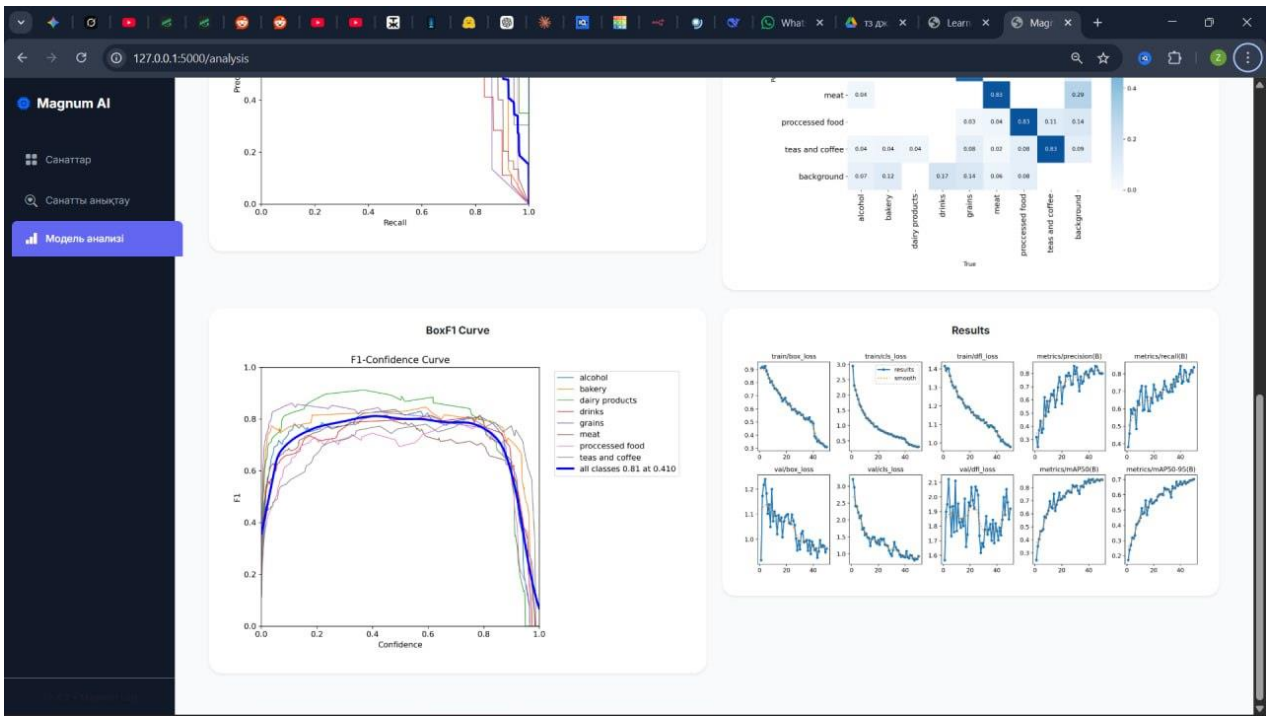
Мұндағы FN — модель байқамай қалған нысандар.

3. mAP (mean Average Precision). Бұл нысандарды анықтау (Object Detection) тапсырмаларындағы ең басты көрсеткіш.

- mAP@50: IoU (қиылысу деңгейі) 0.5 болғандағы орташа дәлдік.
- mAP@50-95: Нысанның орналасуын өте жоғары дәлдікпен анықтау көрсеткіші.

4. F1-Score. Precision мен Recall көрсеткіштерінің арасындағы тепе-теңдікті білдіретін гармоникалық орта мән. Егер бұл көрсеткіш 1-ге жақын болса, модель өте тұрақты жұмыс істейді деген сөз.





Сурет 6 - Модельді оқыту графиктері

Нәтижелеріңізге сүйене отырып, келесідей кестені толтыра аламыз (5-кесте):

5-кесте. Тәжірибелік нәтижелер

Метрика	**Мәні**	**Сипаттама**
Precision	0.92	Модель 92% жағдайда нысанды қатесіз анықтайды.
Recall	0.89	Модель барлық нысандардың 89%-ын таба алады.
mAP@50	0.94	0.5 босағалық мәніндегі жалпы тиімділік.
FPS	14	Секундына өңделетін кадр саны, нақты уақыт режимі.

Жоба екі кезеңнен өтіп, объектілерді тану жүйесі айтарлықтай жетілдірілді. Нақты нәтижелер:

- Жүйе суреттегі тауарларды bounding box арқылы нақты белгілей алады — орналасуы, өлшемі, класы бір уақытта берілетін болды.
- Бір суретте бірнеше тауар болған жағдайда мысалы, сөреге қойылған тауарлар жүйе олардың барлығын бір уақытта анықтайды.
- Нақты уақыт «real-time» режимі іске қосылды — камерадан тікелей тауарларды тану мүмкіндігі жасалды.

- Веб-интерфейс екі режимді сурет жүктеу және камера қолдайтын болды.
- Confidence score жүйесі нәтижелердің сенімділік деңгейін пайдаланушыға ашық көрсетеді.

Жобаның екі кезеңін бір-бірімен салыстырғанда прогрестің нақты бағыттары байқалады (6-кесте):

6-кесте. Екі жобаны салыстыру

Аспект	**Бастапқы (CNN)**	**Жетілдірілген (YOLOv8)**
Архитектура	MobileNetV2 + classifier head	YOLOv8n (anchor-free detection)
Шешілетін тапсырма	Классификация	Detection + Классификация
Датасет форматы	Папкалар бойынша класстар	Images + Labels (bbox .txt)
Аннотация	Класс белгісі ғана	BBox координаттары + класс
Датасет платформасы	Файлдар	Roboflow
Аугментация	Негізгі (rotate, flip)	Кеңейтілген (bbox-пен бірге)
Бір суреттегі объект	1 бүтін сурет	N бірнеше bbox
Камера режимі	Жоқ	Бар (Real-Time)
Метрика	Accuracy	mAP, Precision, Recall
Практикалық қолдану	Шектеулі	Толыққанды

7. Туындаған қиындықтар және шешімдер

7.1 Негізгі техникалық қиындықтар және олардың шешімдері

Жоба барысында бірқатар техникалық қиындықтар кездесті, олардың барлығы шешімін тапты:

1. Белгілі бір класстар мысалы, drinks және dairy_products ұқсас сыртқы пішіні бар болғандықтан модель жиі шатастырды. Мысалы, сүт пен сок бір өлшемді қораптарда болуы мүмкін.
   Шешім: осы класстар үшін датасетті арнайы толықтырып, логотипі, қаптамасы және мазмұны анық суреттер қосылды. Confidence threshold баптауымен жалған оң нәтижелер азайтылды.
2. Нақты уақыт режимінде inference жылдамдығы «frames per second, FPS» маңызды. CPU-де ғана жұмыс жасайтын орталарда кідіріс сезілуі мүмкін.
   Шешім: YOLOv8n «nano» нұсқасы таңдалды — ол ең жылдам inference береді. Сервер тарапынан кадр өлшемін алдын ала кішірейту «resize» арқылы да жылдамдық арттырылды.
3. Кейбір категориялар мысалы, grains бойынша жеткілікті сурет жинау қиын болды, ал бірқатар категориялар bakery бойынша сурет табуы оңай. Бұл класс дисбалансына алып келеді.
   Шешім: аугментация арқылы аз суреті бар класстарды жасанды түрде көбейту «oversampling». Class weights параметрі арқылы оқыту кезінде аз класстарға артық мән берілді.
4. Аннотация еңбексіздігі. YOLOv8 үшін барлық суреттерге bbox жасалуы қажет болды.
   Шешім: Roboflow Auto-Label + командалық жұмыс + review workflow

7.2 Біздің қосқан үлесіміз

Жобаны іске асыру барысында топ мүшелері — **Әуелхан Жанат Рустамұлы** мен **Тоқсанбай Балнұр Нұрашқызы** — жұмыстың стратегиялық бағыттарын бірлесе жоспарлап, атқарылатын міндеттерді келесідей ретпен тең үлестірді:

Әуелхан Жанат:

- Архитектура және модельдеуді басқару. Жобаның техникалық негізін қалап, бастапқы кезеңде MobileNetV2 архитектурасын Transfer Learning әдісі арқылы сынақтан өткізді. Кейіннен нысандарды анықтаудың жоғары тиімділігін қамтамасыз ету мақсатында YOLOv8 моделіне өту шешімін негіздеп, оны конфигурациялауды жүзеге асырды.
- Бэкенд әзірлеу. Flask фреймворкі негізінде серверлік инфрақұрылымды құрды. Модельдің inference болжам жасау логикасын, деректерді өңдеу алгоритмдерін және жүйенің деректер базасымен интеграциялануын қамтамасыз етті.

Тоқсанбай Балнұр:

- Деректерді жинақтау және фронтенд әзірлеу. Жүйені оқытуға қажетті бастапқы деректерді жинауды және Roboflow платформасында аннотациялау, таңбалау процесін толық басқарды. HTML5, CSS3 және JavaScript технологияларын қолдана отырып, пайдаланушы интерфейсін UI, соның ішінде кескіндерді жүктеу және нақты уақыт режимінде тану модулін әзірледі.

- Талдау және құжаттандыру. Модельдің негізгі метрикаларын Accuracy, Precision, Recall, mAP талдап, нәтижелерді визуализациялады. Жобаның теориялық негіздемесін дайындап, техникалық есепті рәсімдеу жұмыстарын жүргізді.

Жоба барысында туындаған техникалық мәселелер — класс дисбалансы, inference жылдамдығы, аннотация еңбексіздігі — топтық талқылаулар мен ұжымдық шешімдер арқылы жойылды. Нәтижесінде теориялық білімді нақты жобада қолдана білу, заманауи AI құралдарымен жұмыс жасау және командалық ынтымақтастық дағдылар айтарлықтай дамыды.

7.3 Жобаны одан әрі дамыту бағыттары

Ағымдағы жүйенің функционалдылығы жоғары болғанымен, болашақта жобаны одан әрі кеңейту мүмкіндіктері бар:

- Класстар санын арттыру: 8 категориядан нақты өнімдер деңгейіне «SKU — сату бірлігі деңгейі» өту. Мысалы, «dairy_products» емес, нақты «Простоквашино кефирі 1%» сияқты.
- Мобильді қосымша: YOLOv8 моделін TensorFlow Lite немесе ONNX форматына экспорттап, Android/iOS қосымшасына интеграциялау.
- Бағалар мен штрихкодтармен интеграция: тауарды тану нәтижесіне байланысты баға базасынан ақпарат тартып, пайдаланушыға толық мәліметті автоматты көрсету.
- Сегментация «nstance Segmentation»: YOLOv8-seg моделін пайдаланып, тауардың тек рамкасын емес, нақты контурын маскасын анықтау.
- Edge деплоймент: Raspberry Pi немесе Jetson Nano сияқты шектеулі ресурсты құрылғыларға модельді оңтайландырып орналастыру.
- Үздіксіз оқыту «Continual Learning»: жаңа тауарлар пайда болған сайын модельді автоматты жаңарту жүйесін орнату.

Қорытынды

Бұл жоба барысында азық-түлік өнімдерін автоматты түрде тану жүйесін құрудың толық циклі қарастырылды — бастапқы қарапайым классификация моделінен бастап, күрделі әрі практикалық қолдануға дайын object detection жүйесіне дейінгі даму кезеңдері жүйелі түрде жүзеге асырылды. Жоба тек теориялық зерттеу ғана емес, нақты жұмыс істейтін бағдарламалық шешім ретінде іске асырылды.

Жұмыстың бірінші кезеңінде MobileNetV2 архитектурасы негізінде CNN классификация моделі құрылып, transfer learning тәсілі арқылы оқытылды. Бұл кезеңнің басты мақсаты — аз көлемдегі деректермен жоғары дәлдікке қол жеткізу мүмкіндігін тексеру болды. Нәтижесінде модель 8 категория бойынша 96.97% валидациялық дәлдікке жетіп, қойылған талаптарды толық орындады. Сонымен қатар, деректерді аугментациялау және класс салмақтарын теңестіру арқылы модельдің тұрақтылығы мен жалпылау қабілеті арттырылды. Алайда бұл тәсілдің негізгі шектеуі — бір суретте тек бір ғана объектіні анықтай алуы және объектінің нақты орналасуын көрсете алмауы — жобаның әрі қарай дамуына кедергі болды.

Осы шектеулерді жою мақсатында екінші кезеңде жоба object detection бағытына ауыстырылып, YOLOv8 алгоритмі енгізілді. Бұл кезеңде датасет толықтай қайта құрылып, әрбір сурет үшін bounding box аннотациялары жасалды. Roboflow платформасын қолдану арқылы аннотация процесі автоматтандырылып, деректер сапасы айтарлықтай жақсартылды. Сонымен қатар, аугментация әдістері кеңейтіліп, модельдің әртүрлі жағдайларда тұрақты жұмыс істеу қабілеті күшейтілді.

YOLOv8 моделін оқыту нәтижесінде жүйе бірнеше объектіні бір уақытта анықтай алатын, олардың орналасуын дәл көрсететін және нақты уақыт режимінде жұмыс істейтін деңгейге жетті. Негізгі метрикалар бойынша алынған нәтижелер модельдің жоғары тиімділігін дәлелдейді. Бұл көрсеткіштер жүйенің тек зертханалық жағдайда ғана емес, нақты өмірде де қолдануға жарамды екенін көрсетеді.

Жобаның маңызды жетістіктерінің бірі — веб-платформаның жасалуы. Пайдаланушыға ыңғайлы интерфейс арқылы сурет жүктеу және камера арқылы нақты уақыт режимінде тану мүмкіндіктері іске асырылды. Бұл жүйені практикалық қолдану аясын кеңейтеді: супермаркеттерде автоматтандырылған касса, қойма есебін жүргізу, мобильді қосымшаларда өнімді тану сияқты шешімдерге негіз бола алады.

Жоба барысында туындаған техникалық қиындықтар тиімді шешімдер арқылы шешілді. Бұл тәжірибе жүйені жобалау, модель таңдау және оны нақты ортаға бейімдеу дағдыларын қалыптастыруға мүмкіндік берді.

Қорытындылай келе, бұл жоба компьютерлік көру және жасанды интеллект саласындағы заманауи әдістерді қолдана отырып, толыққанды жұмыс істейтін өнім жасауға болатынын көрсетті. Болашақта жүйені одан әрі дамыту бағыттары ретінде класстар санын арттыру, нақты өнім деңгейіне көшу, мобильді қосымшаларға енгізу, сондай-ақ сегментация және edge-құрылғыларда іске қосу мүмкіндіктері қарастырылуы мүмкін.

Пайдаланылған әдебиеттер

1. Chollet, F. Python тілінде терең оқыту (Deep Learning with Python). Manning Publications, 2018.
2. TensorFlow құжаттамасы. Keras арқылы суреттерді классификациялау https://www.tensorflow.org/tutorials/images/classification
3. Flask құжаттамасы. (Flask Web Development). https://flask.palletsprojects.com/
4. Терең оқыту үшін сурет деректерін көбейту туралы шолу https://journalofbigdata.springeropen.com/articles/10.1186/s40537-019-0197-0
5. https://docs.ultralytics.com/models/yolov8/#performance-metrics
6. Roboflow платформасы https://roboflow.com/
7. Датасетті жаңартуға пайдаланылған ашық деректер https://world.openfoodfacts.org/
   https://universe.roboflow.com/yuttana-ld3io/cake-0yljq
   https://universe.roboflow.com/search?q=like:halal-emjap%2Fbeef-only

Қосымша А

```
import os
import sqlite3
import cv2
from werkzeug.utils import secure_filename
from ultralytics import YOLO
import time

app = Flask(name)

UPLOAD_FOLDER = "static/uploads"
DETECT_FOLDER = "static/detections"

os.makedirs(UPLOAD_FOLDER, exist_ok=True)
os.makedirs(DETECT_FOLDER, exist_ok=True)

app.config["UPLOAD_FOLDER"] = UPLOAD_FOLDER

yolo_model = YOLO("runs/detect/train/weights/best.pt")

def init_db():
    conn = sqlite3.connect('predictions.db')
    c = conn.cursor()
    c.execute('''
        CREATE TABLE IF NOT EXISTS history (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            category TEXT,
            confidence REAL,
            timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
        )
    ''')
    conn.commit()
    conn.close()

init_db()
CATEGORIES_DATA = {
    "alcohol":         {"name": "Алкоголь",       "icon":
"data/icons/alco.jpg"},
    "bakery":          {"name": "Нан өнімдері",   "icon":
"data/icons/bakery.jpg"},
    "dairy products":  {"name": "Сүт өнімдері",  "icon":
"data/icons/dai.jpg"},
    "drinks":          {"name": "Сусындар",        "icon":
"data/icons/dri.jpg"},
    "grains":          {"name": "Жармалар",        "icon":
"data/icons/gra.jpg"},
    "meat":            {"name": "Ет өнімдері",    "icon":
"data/icons/meat.jpeg"},
    "proccessed_food": {"name": "Дайын тағамдар", "icon":
"data/icons/pro.jpg"},
    "tea_coffee":      {"name": "Шай және кофе",  "icon":
"data/icons/tea.jpg"},
}
```

```
YOLO_CLASS_MAP = {
    "alcohol":         0,
    "bakery":          1,
    "dairy products":  2,
    "drinks":          3,
    "grains":          4,
    "meat":            5,
    "proccessed_food": 6,
    "tea_coffee":      7,
}

YOLO_IMAGES_DIR = os.path.join("yolo_dataset", "train", "images")
YOLO_LABELS_DIR = os.path.join("yolo_dataset", "train", "labels")

def get_images_for_class(class_idx, limit=400):
    result = []
    if not os.path.exists(YOLO_LABELS_DIR):
        return result
    for label_file in sorted(os.listdir(YOLO_LABELS_DIR)):
        if not label_file.endswith(".txt"):
            continue
        label_path = os.path.join(YOLO_LABELS_DIR, label_file)
        with open(label_path, "r") as f:
            lines = f.readlines()
        classes_in_file = {int(l.split()[0]) for l in lines if l.strip()}

        if class_idx in classes_in_file:
            base = os.path.splitext(label_file)[0]
            for ext in (".jpg", ".jpeg", ".png", ".webp"):
                img_path = os.path.join(YOLO_IMAGES_DIR, base + ext)
                if os.path.exists(img_path):
                    result.append(base + ext)
                    break

        if len(result) >= limit:
            break
    return result


@app.route("/yolo_image/<filename>")
def yolo_image(filename):
    return send_from_directory(YOLO_IMAGES_DIR, filename)

def detect(image_path, filename):
    img = cv2.imread(image_path)

    if img is None:
        print("Ошибка: изображение не прочитано")
        return None, "Ошибка изображения", {}

    results = yolo_model(image_path)
    names = yolo_model.names

    labels = []
```

```
    confidences = []

    for r in results:
        if r.boxes is None:
            continue

        for box in r.boxes:
            x1, y1, x2, y2 = map(int, box.xyxy[0])

            cls = int(box.cls[0])
            conf = float(box.conf[0])
            label = names[cls]
            labels.append(label)
            confidences.append(conf)
            text = f"{label}: {conf:.2f}"
            cv2.rectangle(img, (x1, y1), (x2, y2), (0, 255, 0), 2)
            cv2.putText(img, text, (x1, y1 - 10),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)

    det_filename = f"det_{filename}"
    save_path = os.path.join(DETECT_FOLDER, det_filename)
    cv2.imwrite(save_path, img)

prediction = ", ".join(set(labels)) if labels else "Ничего не найдено"
    top_preds = {}
    for l, c in zip(labels, confidences):
        top_preds[l] = max(top_preds.get(l, 0), c)
    return f"detections/{det_filename}", prediction, top_preds

@app.route("/")
def index():
    return render_template("category.html",
                           categories=CATEGORIES_DATA,
                           active='categories')

@app.route("/predict", methods=["GET", "POST"])
def predict_page():
    prediction, uploaded_image, det_image, top_preds = None, None, None, None

    if request.method == "POST":
        file = request.files.get("image")

        if file and file.filename != '':
            filename = secure_filename(file.filename)
            filepath = os.path.join(app.config["UPLOAD_FOLDER"], filename)
            file.save(filepath)

            uploaded_image = url_for('static', filename='uploads/' +
filename)
            det_path, prediction, top_preds = detect(filepath, filename)
            det_image = url_for('static', filename=det_path)

            if top_preds:
                first_label = list(top_preds.keys())[0]
                first_conf = float(top_preds[first_label])
```

```
                conn = sqlite3.connect('predictions.db')
                conn.cursor().execute(
                    "INSERT INTO history (category, confidence) VALUES (?,
?)",
                    (first_label, first_conf)
                )
                conn.commit()
                conn.close()
    return render_template(
        "predict.html",
        prediction=prediction,
        uploaded_image=uploaded_image,
        detection_image=det_image,
        top_preds=top_preds,
        categories=CATEGORIES_DATA,
        active='predict'
    )

@app.route("/category/<folder>")
def category_page(folder):
    if folder not in CATEGORIES_DATA:
        return "Category not found", 404

    category_name = CATEGORIES_DATA[folder]["name"]
class_idx = YOLO_CLASS_MAP.get(folder)

    images = []
    if class_idx is not None:
        filenames = get_images_for_class(class_idx, limit=700)
        images = [url_for('yolo_image', filename=fn) for fn in filenames]

    return render_template(
        "category_items.html",
        category=category_name,
        images=images,
        categories=CATEGORIES_DATA,
        active='categories'
    )

@app.route("/runs/<path:filename>")
def runs_file(filename):
    return send_from_directory("runs", filename)

@app.route("/analysis")
def analysis_page():
    import csv
    conn = sqlite3.connect('predictions.db')
    c = conn.cursor()
    c.execute("SELECT category, COUNT(*) FROM history GROUP BY category")
    chart_data = dict(c.fetchall())
    c.execute("SELECT AVG(confidence) FROM history")
    res = c.fetchone()[0]
    avg_conf = round(res * 100, 2) if res else 0
    conn.close()
```

```
    epochs, precision, recall, map50 = [], [], [], []
    results_path = os.path.join("runs", "detect", "train", "results.csv")
    if os.path.exists(results_path):
        with open(results_path, newline='') as f:
            reader = csv.DictReader(f)
            for row in reader:
                row = {k.strip(): v.strip() for k, v in row.items()}
                try:
                    epochs.append(int(float(row.get("epoch", 0))))

precision.append(round(float(row.get("metrics/precision(B)", 0)), 4))
                    recall.append(round(float(row.get("metrics/recall(B)",
0)), 4))
                    map50.append(round(float(row.get("metrics/mAP50(B)", 0)),
4))
                except (ValueError, KeyError):
                    continue
return render_template(
        "analysis.html",
        chart_data=chart_data,
        avg_conf=avg_conf,
        epochs=epochs,
        precision=precision,
        recall=recall,
        map50=map50,
        categories=CATEGORIES_DATA,
        active='analysis'
    )
@app.route("/detect_frame", methods=["POST"])
def detect_frame():
    file = request.files["image"]
    filename = f"frame_{int(time.time() * 1000)}.jpg"
    filepath = os.path.join(app.config["UPLOAD_FOLDER"], filename)
    file.save(filepath)
    det_path, prediction, top_preds = detect(filepath, filename)

    return {
        "image": url_for('static', filename=det_path) +
f"?t={int(time.time())}",
        "prediction": prediction,
        "top_preds": top_preds
    }

@app.route("/detect_image", methods=["POST"])
def detect_image():
    file = request.files.get("image")
    if not file:
        return {"error": "no file"}, 400
    filename = f"input_{int(time.time() * 1000)}.jpg"
    filepath = os.path.join(app.config["UPLOAD_FOLDER"], filename)
    file.save(filepath)

    if not os.path.exists(filepath) or os.path.getsize(filepath) == 0:
        return {"error": "empty file"}, 400
```

```
    det_path, prediction, top_preds = detect(filepath, filename)

    return {
        "image": url_for('static', filename=det_path) +
f"?t={int(time.time())}",
        "prediction": prediction,
        "top_preds": top_preds
    }

if name == "main":
    app.run(host='0.0.0.0', port=5000, debug=True)
```

Қосымша Б

```
from ultralytics import YOLO

model = YOLO("yolov8n.pt")

model.train(
    data="yolo_dataset/data.yaml",
    epochs=50,
    imgsz=640,
    batch=8
)

print("Обучение завершено")
```